DATA 412/612 · Statistical Programming in R · Week 14

# Functional Programming with purrr: 10 Examples

Ten examples from map() to walk(), each with a worked example and a solution.

Apply the same operation to each piece: a vector, a list, the columns of a data frame, a group, or several inputs side by side.

Datasets: `iris`, `mtcars`, `airquality`, `ToothGrowth`, `ChickWeight`.

Run the worked example. Then write Your turn before you read the solution. Some exercises use different columns from the worked example.

```
set.seed(612)
library(tidyverse)
```

---

## 1. A list and `map()`

`mtcars`. Split a column, then apply one function to each piece.

### Worked example

```
mpg_by_cyl <- split(mtcars$mpg, mtcars$cyl)
str(mpg_by_cyl)
```

```
## List of 3
##  $ 4: num [1:11] 22.8 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26 30.4 ...
##  $ 6: num [1:7] 21 21 21.4 18.1 19.2 17.8 19.7
##  $ 8: num [1:14] 18.7 14.3 16.4 17.3 15.2 10.4 10.4 14.7 15.5 15.2 ...
```

```
map(mpg_by_cyl, mean)
```

```
## $`4`
## [1] 26.66364
##
## $`6`
## [1] 19.74286
##
## $`8`
## [1] 15.1
```

```
map_dbl(mpg_by_cyl, mean)
```

```
##        4        6        8
## 26.66364 19.74286 15.10000
```

`map()` applies `mean` to the 4-, 6-, and 8-cylinder groups. Pass `mean` without parentheses.

### Your turn

Split `mtcars$hp` by `mtcars$gear`. Map `max`. Repeat with `map_dbl()`. What changes?

### Solution

```
hp_by_gear <- split(mtcars$hp, mtcars$gear)
map(hp_by_gear, max)
```

```
## $`3`
## [1] 245
##
## $`4`
## [1] 123
##
## $`5`
## [1] 335
```

```
map_dbl(hp_by_gear, max)
```

```
##   3   4   5
## 245 123 335
```

`map()` returns a list. `map_dbl()` returns a double vector. Each group must produce one numeric value.

---

## 2. `map()` on data-frame columns

`iris`. A data frame is a list of columns, so `map()` walks the columns.

### Worked example

```
map(iris, class)
```

```
## $Sepal.Length
## [1] "numeric"
##
## $Sepal.Width
## [1] "numeric"
##
## $Petal.Length
## [1] "numeric"
##
## $Petal.Width
## [1] "numeric"
##
## $Species
## [1] "factor"
```

```
iris |>
  select(where(is.numeric)) |>
  map_dbl(sd)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##    0.8280661    0.4358663    1.7652982    0.7622377
```

`Species` is a factor. Select the numeric columns before `sd()`.

### Your turn

Median of every numeric `iris` column, as one named numeric vector. Class of each column, as a character vector. Why does `map_chr(iris, class)` work here?

### Solution

```
iris |>
  select(where(is.numeric)) |>
  map_dbl(median)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##         5.80         3.00         4.35         1.30
```

```
map_chr(iris, class)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##    "numeric"    "numeric"    "numeric"    "numeric"     "factor"
```

`class()` produces one string per column here. `map_chr()` requires exactly one character value per element.

---

## 3. Missing values and a lambda

`airquality`. `\(x)` is short for `function(x)`, from R 4.1.

### Worked example

```
map_lgl(airquality, \(x) any(is.na(x)))
```

```
##   Ozone Solar.R    Wind    Temp   Month     Day
##    TRUE    TRUE   FALSE   FALSE   FALSE   FALSE
```

```
map_int(airquality, \(x) sum(is.na(x)))
```

```
##   Ozone Solar.R    Wind    Temp   Month     Day
##      37       7       0       0       0       0
```

The first call is logical. The second is an integer count.

### Your turn

Proportion missing in every column, with `map_dbl()`. Names of the columns that have any missing value. Mean of `Ozone`, ignoring missing values.

### Solution

```
missing_proportion <- map_dbl(airquality, \(x) mean(is.na(x)))
missing_proportion
```

```
##      Ozone    Solar.R       Wind       Temp      Month        Day
## 0.24183007 0.04575163 0.00000000 0.00000000 0.00000000 0.00000000
```

```
missing_any <- map_lgl(airquality, \(x) any(is.na(x)))
names(missing_any)[missing_any]
```

```
## [1] "Ozone"   "Solar.R"
```

```
mean(airquality$Ozone, na.rm = TRUE)
```

```
## [1] 42.12931
```

```
map_dbl(airquality["Ozone"], \(x) mean(x, na.rm = TRUE))
```

```
##    Ozone
## 42.12931
```

`mean(is.na(x))` is the fraction of `TRUE`, because `TRUE` is 1 and `FALSE` is 0.

---

## 4. `across()`

`iris`. `across()` inside `summarise()` keeps one row per species. `map()` on a data frame does not keep the grouping column.

### Worked example

```
iris |>
  group_by(Species) |>
  summarise(
    across(
      ends_with("Length"),
      list(mean = mean, sd = sd),
      .names = "{.col}_{.fn}"
    ),
    .groups = "drop"
  )
```

```
## # A tibble: 3 × 5
##   Species    Sepal.Length_mean Sepal.Length_sd Petal.Length_mean Petal.Length_sd
##   <fct>                  <dbl>           <dbl>             <dbl>           <dbl>
## 1 setosa                  5.01           0.352              1.46           0.174
## 2 versicolor              5.94           0.516              4.26           0.470
## 3 virginica               6.59           0.636              5.55           0.552
```

### Your turn

For each species, the median and the IQR of `Sepal.Width` and `Petal.Width`. Names such as `Sepal.Width_median` and `Sepal.Width_IQR`.

### Solution

```
iris |>
  group_by(Species) |>
  summarise(
    across(
      c(Sepal.Width, Petal.Width),
      list(median = median, IQR = IQR),
      .names = "{.col}_{.fn}"
    ),
    .groups = "drop"
  )
```

```
## # A tibble: 3 × 5
##   Species  Sepal.Width_median Sepal.Width_IQR Petal.Width_median Petal.Width_IQR
##   <fct>                 <dbl>           <dbl>              <dbl>           <dbl>
## 1 setosa                  3.4           0.475                0.2             0.1
## 2 versico…                2.8           0.475                1.3             0.3
## 3 virgini…                3             0.375                2               0.5
```

One row for each of the three species.

---

## 5. One function, many groups

`ToothGrowth`. A named function can return a one-row data frame. `map()` then returns a list of those data frames.

`map_dfr()` and `map_dfc()` still run and purrr marks them superseded; collect data frames with `map(...) |> list_rbind()` or `list_cbind()`.

### Worked example

```
mean_sd <- function(x) {
  tibble(mean = mean(x, na.rm = TRUE), sd = sd(x, na.rm = TRUE))
}
length_by_supp <- split(ToothGrowth$len, ToothGrowth$supp)
results <- map(length_by_supp, mean_sd)
results
```

```
## $OJ
## # A tibble: 1 × 2
##    mean    sd
##   <dbl> <dbl>
## 1  20.7  6.61
##
## $VC
## # A tibble: 1 × 2
##    mean    sd
##   <dbl> <dbl>
## 1  17.0  8.27
```

```
bind_rows(results, .id = "supp")
```

```
## # A tibble: 2 × 3
##   supp   mean    sd
##   <chr> <dbl> <dbl>
## 1 OJ     20.7  6.61
## 2 VC     17.0  8.27
```

`.id` puts the list names `OJ` and `VC` into `supp`.

### Your turn

Write `five_number()` so it returns a one-row tibble with the minimum, the median, and the maximum. Apply it to `ToothGrowth$len` split by `dose`. Combine the rows and keep a `dose` column.

### Solution

```
five_number <- function(x) {
  tibble(minimum = min(x), median = median(x), maximum = max(x))
}
ToothGrowth$len |>
  split(ToothGrowth$dose) |>
  map(five_number) |>
  bind_rows(.id = "dose")
```

```
## # A tibble: 3 × 4
##   dose  minimum median maximum
##   <chr>   <dbl>  <dbl>   <dbl>
## 1 0.5       4.2   9.85    21.5
## 2 1        13.6  19.2     27.3
## 3 2        18.5  26.0     33.9
```

`dose` is character because it was a list name. Use `as.numeric(dose)` when you need a number.

---

## 6. `map2()`

`mtcars`. Two vectors, paired by position.

### Worked example

```
mpg <- mtcars$mpg[1:5]
distance <- c(100, 150, 80, 120, 200)
price_per_gallon <- 3.50
cost <- map2_dbl(distance, mpg, \(d, m) (d / m) * price_per_gallon)
tibble(car = rownames(mtcars)[1:5], distance, mpg, cost)
```

```
## # A tibble: 5 × 4
##   car               distance   mpg  cost
##   <chr>                <dbl> <dbl> <dbl>
## 1 Mazda RX4              100  21    16.7
## 2 Mazda RX4 Wag          150  21    25
## 3 Datsun 710              80  22.8  12.3
## 4 Hornet 4 Drive         120  21.4  19.6
## 5 Hornet Sportabout      200  18.7  37.4
```

### Your turn

The first five cars. Distances `c(50, 75, 100, 125, 150)`. Four dollars a gallon. `map2_dbl()` for the fuel cost. A tibble with the car name and the fuel cost.

### Solution

```
d <- c(50, 75, 100, 125, 150)
m <- mtcars$mpg[1:5]
fuel_cost <- map2_dbl(d, m, \(distance, mpg) distance / mpg * 4)
tibble(car = rownames(mtcars)[1:5], fuel_cost)
```

```
## # A tibble: 5 × 2
##   car               fuel_cost
##   <chr>                 <dbl>
## 1 Mazda RX4              9.52
## 2 Mazda RX4 Wag         14.3
## 3 Datsun 710            17.5
## 4 Hornet 4 Drive        23.4
## 5 Hornet Sportabout     32.1
```

---

## 7. `pmap()`

`mtcars`. Three inputs, still paired by position.

## Worked example

```
trips <- tibble(
  car = rownames(mtcars)[1:4],
  distance = c(50, 100, 150, 200),
  mpg = mtcars$mpg[1:4],
  price = c(3.25, 3.50, 3.75, 4.00)
)
trips |>
  mutate(
    cost = pmap_dbl(
      list(distance, mpg, price),
      \(distance, mpg, price) distance / mpg * price
    )
  )
```

```
## # A tibble: 4 × 5
##   car            distance   mpg price  cost
##   <chr>             <dbl> <dbl> <dbl> <dbl>
## 1 Mazda RX4            50  21    3.25  7.74
## 2 Mazda RX4 Wag       100  21    3.5  16.7
## 3 Datsun 710          150  22.8  3.75 24.7
## 4 Hornet 4 Drive      200  21.4  4    37.4
```

## Your turn

A shipping tibble: `weight = c(2, 5, 8)`, `rate = c(1.5, 2, 2.5)`, `base_fee = c(3, 4, 5)`. `pmap_dbl()` for `base_fee + weight * rate`.

## Solution

```
shipping <- tibble(
  weight = c(2, 5, 8),
  rate = c(1.5, 2, 2.5),
  base_fee = c(3, 4, 5)
)
shipping |>
  mutate(
    cost = pmap_dbl(
      list(base_fee, weight, rate),
      \(base_fee, weight, rate) base_fee + weight * rate
    )
  )
```

```
## # A tibble: 3 × 4
##   weight  rate base_fee  cost
##    <dbl> <dbl>    <dbl> <dbl>
## 1      2   1.5        3     6
## 2      5   2          4    14
## 3      8   2.5        5    25
```

The costs are 6, 14, and 25.

---

## 8. `group_map()` and `group_modify()`

`ChickWeight`. `data` is the group's rows. `key` is the group id. `group_map()` returns a list. `group_modify()` binds the data frames and keeps the group column.

### Worked example

```
fit_slope <- function(data) {
  fit <- lm(weight ~ Time, data = data)
  tibble(slope = unname(coef(fit)["Time"]))
}
ChickWeight |>
  group_by(Diet) |>
  group_map(\(data, key) fit_slope(data))
```

```
## [[1]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  6.84
##
## [[2]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  8.61
##
## [[3]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  11.4
##
## [[4]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  9.71
```

```
ChickWeight |>
  group_by(Diet) |>
  group_modify(\(data, key) fit_slope(data)) |>
  ungroup()
```

```
## # A tibble: 4 × 2
##   Diet  slope
##   <fct> <dbl>
## 1 1      6.84
## 2 2      8.61
## 3 3     11.4
## 4 4      9.71
```

### Your turn

`mtcars`. Fit `mpg ~ wt` inside each `cyl` group. One row per cylinder group, with the slope on `wt`. Which group has the most negative slope?

### Solution

```
model_slopes <- mtcars |>
  group_by(cyl) |>
  group_modify(\(data, key) {
    model <- lm(mpg ~ wt, data = data)
    tibble(wt_slope = unname(coef(model)["wt"]))
  }) |>
  ungroup()
model_slopes
```

```
## # A tibble: 3 × 2
##     cyl wt_slope
##   <dbl>    <dbl>
## 1     4    -5.65
## 2     6    -2.78
## 3     8    -2.19
```

```
model_slopes |>
  slice_min(wt_slope, n = 1)
```

```
## # A tibble: 1 × 2
##     cyl wt_slope
##   <dbl>    <dbl>
## 1     4    -5.65
```

A more negative slope means fitted MPG falls faster as weight rises inside that cylinder group. That is an association, not a cause.

## 9. `imap()`

`iris`. `imap()` passes the element and its name.

### Worked example

```
numeric_iris <- iris |>
  select(where(is.numeric))
column_reports <- imap_chr(
  numeric_iris,
  \(values, column_name) paste0(column_name, ": mean = ", round(mean(values), 2))
)
column_reports
```

```
##                Sepal.Length                 Sepal.Width
## "Sepal.Length: mean = 5.84"  "Sepal.Width: mean = 3.06"
##                Petal.Length                 Petal.Width
## "Petal.Length: mean = 3.76"   "Petal.Width: mean = 1.2"
```

### Your turn

`imap_chr()` on the numeric columns of `mtcars`. Strings like `mpg: min = 10.4`.

### Solution

```
mtcars |>
  select(where(is.numeric)) |>
  imap_chr(\(values, column_name) paste0(column_name, ": min = ", min(values)))
```

```
##                mpg                cyl               disp                 hp
##  "mpg: min = 10.4"     "cyl: min = 4" "disp: min = 71.1"     "hp: min = 52"
##               drat                 wt               qsec                 vs
## "drat: min = 2.76"  "wt: min = 1.513" "qsec: min = 14.5"      "vs: min = 0"
##                 am               gear               carb
##      "am: min = 0"    "gear: min = 3"    "carb: min = 1"
```

---

## 10. `walk()` for a side effect

`iris`. `walk()` is for a file or a plot. It returns the input invisibly. The files below are written under `tempdir()`, not in the project folder.

### Worked example

```
plot_dir <- file.path(tempdir(), "data612_iris_plots")
unlink(plot_dir, recursive = TRUE)
dir.create(plot_dir)
plot_vars <- c("Sepal.Length", "Sepal.Width", "Petal.Length")
walk(plot_vars, \(var) {
  p <- ggplot(iris, aes(x = .data[[var]])) +
    geom_histogram(bins = 15)
  ggsave(
    filename = file.path(plot_dir, paste0(var, ".png")),
    plot = p,
    width = 5,
    height = 3.5,
    units = "in"
  )
})
list.files(plot_dir)
```

```
## [1] "Petal.Length.png" "Sepal.Length.png" "Sepal.Width.png"
```

### Your turn

Four histogram PNGs, one per numeric `iris` column, in a different temporary directory. Check that four PNG files are there.

### Solution

```
out_dir <- file.path(tempdir(), "data612_iris_numeric")
unlink(out_dir, recursive = TRUE)
dir.create(out_dir)
numeric_cols <- names(iris)[map_lgl(iris, is.numeric)]
walk(numeric_cols, \(var) {
  p <- ggplot(iris, aes(x = .data[[var]])) +
    geom_histogram(bins = 15)
  ggsave(
    filename = file.path(out_dir, paste0(var, ".png")),
    plot = p,
    width = 5,
    height = 3.5,
    units = "in"
  )
})
saved_files <- list.files(out_dir)
saved_files
```

```
## [1] "Petal.Length.png" "Petal.Width.png"  "Sepal.Length.png" "Sepal.Width.png"
```

```
length(saved_files)
```

```
## [1] 4
```

## Quick reference

| Goal | Function | Output |
|---|---|---|
| Apply one function to each element | `map()` | List |
| Return one number per element | `map_dbl()` | Double vector |
| Return one integer per element | `map_int()` | Integer vector |
| Return one string per element | `map_chr()` | Character vector |
| Return one `TRUE` / `FALSE` per element | `map_lgl()` | Logical vector |
| Apply functions across selected columns | `across()` | Depends on `mutate()` / `summarise()` |
| Iterate over two inputs | `map2()` / `map2_dbl()` | List / double vector |
| Iterate over many inputs | `pmap()` / `pmap_dbl()` | List / double vector |
| Work with elements and their names | `imap()` / `imap_chr()` | List / character vector |
| Return a list of grouped results | `group_map()` | List |
| Combine data frames from groups | `group_modify()` | Data frame |
| Write files or generate plots | `walk()` | The input, invisibly |

## Concept check

Why is `map_dbl()` often preferable to `map()` when every iteration must produce a single number?

Why is `walk()` more appropriate than `map()` when saving images?